

FC Architecture Comparison — UAVXArm32F4 vs UAVXArmQ

Overview

This document compares the architectural differences between the original UAVXArm32F4 firmware and the UAVXArmQ fork. UAVXArmQ is not an incremental update — it reworks core subsystems while maintaining wire-protocol compatibility with the GCS.

1. Attitude Representation

UAVXArm32F4

Euler-angle / DCM attitude representation. The Madgwick filter outputs roll, pitch, and yaw angles directly. The control pipeline converts setpoint angles to errors using Euler subtraction, which requires trigonometric conversions at each iteration and suffers from gimbal lock at $\pm 90^\circ$ pitch.

UAVXArmQ

Full quaternion attitude representation throughout the estimation and control pipeline:

- **Estimator output:** Madgwick filter produces a unit quaternion. The optional Kalman filter path also uses quaternion state.
- **Control input:** Per-axis `ROLL_ANGLE_Q_KP`, `PITCH_ANGLE_Q_KP`, `YAW_ANGLE_Q_KP` (tags 2/7/96) scale quaternion error directly as $2 \cdot Q_a \cdot P.K_p$. A single global `QUAT_GAIN` (tag 75) was tried first but replaced; it now targets `gUnused` and has no effect. The quaternion error $q_{desired} \times q_{current}^{-1}$ provides a singularity-free rotation vector.
- **Telemetry output only:** `Quat2Euler()` is called once per telemetry cycle for GCS display. The control loop never touches Euler angles.

Why: Eliminates gimbal lock, reduces per-iteration trig calls, enables smooth setpoint interpolation via SLERP, and simplifies the control math to quaternion multiply (4 mul + 3 add) vs. Euler-angle trig chains.

2. Control Pipeline

Component	UAVXArm32F4	UAVXArmQ
Angle controller	Per-axis P from Euler error	Quaternion error $\times$ per-axis P.Kp
Rate controller	PID on gyro rates	PID on gyro rates (unchanged)
Mixer	Axis mixing after Euler→rotation	Quaternion-rotated axis mixing
Yaw handling	Separate yaw path	Unified quaternion yaw
Feedforward	None	Quaternion-rate feedforward

The rate controller structure (PID on body-axis gyro rates) is preserved from the original. The key change is upstream: the angle controller now receives a quaternion error vector rather than three independent Euler errors, which inherently couples the axes correctly during aggressive manoeuvres.

3. Altitude Hold Architecture

UAVXArm32F4

Two-stage cascaded controller:

ALT_POS_Kp → desired climb rate → ALT_VEL_Kp → throttle offset

ALT_VEL_Ki (integrator)

Parameters: ALT_POS_KP, ALT_POS_KI, ALT_VEL_KP, ALT_VEL_KI, ALT_VEL_INT_LIMIT.

UAVXArmQ

Single-stage position PID with cruise throttle feedforward:

ALT_POS_Kp → throttle offset (+ cruise feedforward)

ALT POS Ki ↗

Parameters: ALT_P0S_KP, ALT_P0S_KI, ALT_P0S_INT_LIMIT only. ALT_VEL_KP, ALT_VEL_KI, ALT_VEL_INT_LIMIT are UNUSED (tag slots preserved for index alignment).

Why: The velocity loop was designed for a barometer-only altitude estimate with significant lag. The GPS+baro estimator and cruise feedforward make it redundant. Removing it eliminates a derivative path that amplified barometer noise and simplifies tuning to a single Kp/Ki pair.

Cruise Throttle Feedforward

TrackCruiseThrottle() learns the hover throttle by tracking AltHoldThrComp (the feedback PI term). The learned value is stored in Config.ParamData[59].f (pCruiseThrottle) and is persisted across power cycles.

The original formulation tracked $\text{DesiredThrottle} + \text{AltHoldThrComp} - \text{pCruiseThrFF}$, which was sensitive to rate-of-climb noise. Tracking AltHoldThrComp directly gives a cleaner signal. The tracking rate was increased from 0.5 %/s to 2 %/s for faster convergence.

4. Parameter System

UAVXArm32F4

Mixed uint8 / float32 storage. Parameters were stored as uint8 in `Config.ParamData[i].b` for legacy parameters and as float32 in `.f` for newer ones. The GCS tracked which format each index used via `LEGACY_TAGS` and `PARAM_SCALES` tables. Wire protocol required the GCS to know the format to read/write correctly.

UAVXArmQ

Unified float32: All 128 parameters stored as IEEE-754 float32 in `Config.ParamData[i].f`. The `.b` field and all GCS conversion tables are removed.

Aspect	UAVXArm32F4	UAVXArmQ
Storage	Mixed uint8/float32	All float32
Wire format	Format-dependent	4-byte float32 every time
GCS conversion	LEGACY_TAGS + SCALES	PARAM_DISPLAY_MULT only
Dynamic range	0-255 vs. $\pm 1e38$	$\pm 1e38$ for all

Aspect	UAVXArm32F4	UAVXArmQ
Black-box logging	Variable-size	Uniform 4-byte per param

Wire Protocol (same on both)

- Tag 17: Individual write — `index(1) + float32(4)`
- Tag 71: Bulk readback — `base_idx(1) + count(1) + float32×count`
- Display: `display = fc_float × PARAM_DISPLAY_MULT[idx]`
- Write: `fc_float = display / PARAM_DISPLAY_MULT[idx]`

The protocol bytes are identical; only the FC's internal storage and the GCS's display logic differ.

5. Telemetry Architecture

Packet	UAVXArm32F4	UAVXArmQ
Attitude (tag 68)	Per-axis, cycled 2.5 Hz	Removed from control loop
Tuning (tag 57)	Not present	22-byte all-axes packet, 50 Hz
Config name (tag 63)	Broadcast 25 Hz	Request-only
Config packet	Full config dump	Same

Tuning Packet (UAVXArmQ only)

`SendTuningPacket()` emits at every telemetry cycle (~50 Hz):

`flags(1) + throttle(1) + 3 × [desired_rate(2) + actual_rate(2) + axis_out(2)]`

- All three axes every cycle (vs. one axis per call in legacy)
- Uniform 16-bit precision for all numeric fields
- Includes throttle and mode flags for context
- Not gated behind `pBBLogType`

Config Name

`SendConfigPacket()` (tag 63) was changed from broadcast (every other telemetry cycle, ~25 Hz) to request-only. The GCS requests it once on connect via subtag `UAVXAFNamePacketTag`. This saves ~30 bytes/s of link bandwidth for a static string.

6. Safety Subsystems

Arming — UAVXArm32F4

Arming logic distributed across `rc.c` with separate checks for switch arming and TX arming. Channel 6 (`NavQualificationRC`) used for arming qualification.

Arming — UAVXArmQ

Same hardware mapping (Ch6 → NavQualificationRC) but with cleaner separation:

Method	Arm	AltHold	WPNav
Switch Tx (pot)	Physical Ch6 Ch6 > 10 %	Always on > 40 %	Always on > 60 %

All `F.AltControlEnabled` = assignments consolidated to `MapRC()` in `rc.c`.

Failsafe — UAVXArm32F4

Two paths: 1. SBus failsafe flag → `RCFailsafe` → FS presets 2. Timeout → `GenerateFail-safePacket()` → `RCInp` + `RCFailsafe`

Failsafe — UAVXArmQ

Same two paths, same FS presets (NavQual=1650, NavMode=2000/RTH, sticks=1500, throttle=1050). Key enhancements:

- `RCFailsafe` || `RCSignalLost` at end of NavQual block forces AngleMode + AltHold regardless of switch position
- `PassThru` overrides all failsafe behaviour for fixed-wing manual recovery
- Configurable failsafe action planned (param enum: RTH/Land/MotorsOff)

Dive Recovery (UAVXArmQ only)

`DiveRecover()` in `control.c` detects rapid uncommanded altitude loss (exceeds `DIVE_RECOVER_ALT` meters within 500 ms) and applies full throttle with leveling. Activated by `DIVE_RECOVER_ALT` (tag 116).

7. Emulation

UAVXArm32F4

Emulation constants in `emu.h` were generic, not matching any specific airframe. The GCS emulation mode was usable for functional testing but did not represent real flight behaviour.

UAVXArmQ

Constants updated to match the Ecks 220mm airframe: - `EM_THR_CRUISE_STICK` = 0.55 (55 % hover) - `EM_MASS` = 0.8 (800 g) - `EM_MODEL_HEAVY`: 0.60 / 1.0 kg

Accurate emulation makes the GCS emulation mode representative of actual flight for pre-flight tuning validation.

8. Airframe Files (.af)

Both forks support .af files in the same format. UAVXArmQ introduces a unified-float format where all values are stored in FC-native units (radians, volts, fractions — no scaled integers). The GCS auto-selects Ecks_800g_Moderate.af on first launch.

The .af format is backward-compatible: the GCS reads/writes .af files the same way for both forks, but UAVXArmQ's files use float32-native values while UAVXArm32F4 files may still contain legacy uint8-derived values.

9. Source File Comparison

Subsystem	UAVXArm32F4	UAVXArmQ
Attitude estimation	'ahrs.c'	'ahrs.c' (quaternion)
Attitude control	'control.c' (Euler)	'control.c' (quaternion)
Altitude hold	'altitude.c' (cascade)	'altitude.c' (single-stage + ff)
Telemetry	'telem.c' (per-axis)	'telem.c' (tuning packet)
Parameters	'params.c' (mixed)	'params.c' (unified float)
Dive recovery	Not present	'control.c' (DiveRecover)
Emulation	'emu.h' (generic)	'emu.h' (Ecks-specific)
RC/Arming	'rc.c'	'rc.c' (consolidated)
Failsafe	'rc.c'	'rc.c' (enhanced override)